Fault Tolerant System Design
Lecture 4: Hardware Redundancy

Seyedhamidreza Mousavi

Outlines

- Hardware redundancy
  - Passive redundancy techniques
  - Active redundancy techniques
  - Hybrid redundancy techniques

Redundancy

- 2nd CPU, 2nd ALU, …
  - hardware redundancy
- validation test, …
  - software redundancy
- error-detecting and correcting codes
  - information redundancy
- repeating tasks several times
  - time redundancy

Redundancy

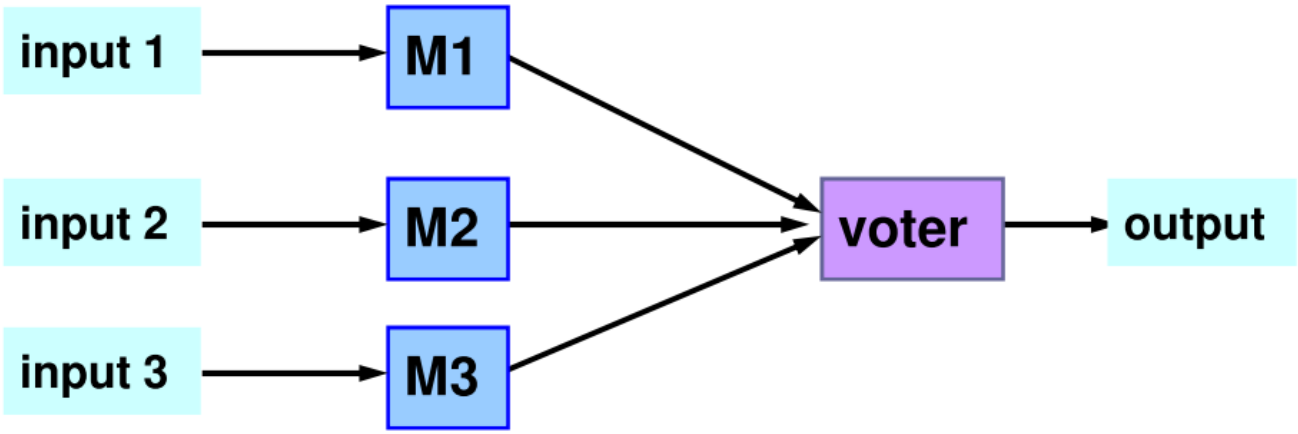
- NOTHING FOR FREE!
- costs
  - HW: components, area, power, ...
  - SW: development costs, ...
  - information: extra HW to code / decode
  - time: faster CPUs, components
- trade-off against increase in dependability

HW redundancy: overview

- passive redundancy techniques (static)
  - fault masking
- active redundancy techniques (dynamic)
  - detection, localization, containment, recovery
- hybrid redundancy techniques
  - static + dynamic
  - fault masking + reconfiguration

Passive HW redundancy

- Triple Modular Redundancy (TMR)



Passive HW redundancy

- Triple Modular Redundancy (TMR)
  - 3 active components
  - fault masking by voter
- Problem: voter is a single point of failure

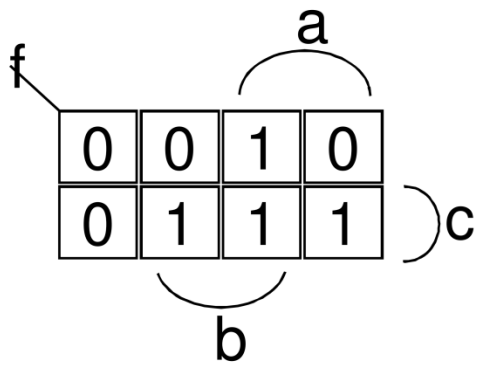
Passive HW redundancy

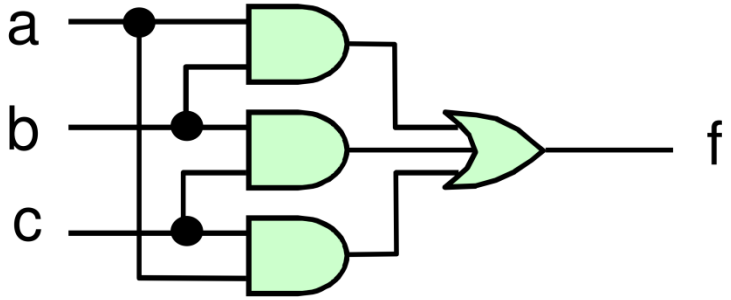
- N-modular redundancy (NMR)
  - N active components (N A)
  - N odd, for majority voting
  - tolerates [N/2] module faults
- example Apollo
  - N=5
  - 2 faults can be tolerated (masked)

HW voting

hardware realisation of 1-bit majority voter

f = ab + ac + bc





n-bit majority voter: n times 1-bit

requires 2 gate delays

SW voting

- Voting can be performed using software
- voter is software implemented by a microprocessor
- voting program can be as simple as a sequence of three comparisons, with the outcome of the vote being the value that agrees with at least on the other two

HW vs. SW Voting

- HW: fast, but expensive
  - 32-bit voter: 128 gates and 256 flip flops
  - 1 TMR level = 3 voters
- SW: slow, but more flexible
  - use existing CPUs

Problem with voting

- Major problem with practical application of voting is that the three results may not completely agree
  - sensors, used in many control systems, can seldom be manufactured so that their values agree exactly
  - analog-to-digital converter can produce quantities that disagree in the least significant bits

Problems with voting

- (1) When values that disagree slightly are processed, the disagreement can grow larger
  - small difference in inputs can produce large differences in outputs
- (2) A single result must ultimately be produced
  - potential point where one failure can cause a system failure

How to cure problem 1

- Ignore the least-significant bits of data
  - disagreement which occurs only in the least-significant bits is acceptable
  - disagreement which affects the most-significant bits is not acceptable and must be corrected

HW redundancy: overview

- passive redundancy techniques (static)
  - fault masking
- active redundancy techniques (dynamic)
  - detection, localization, containment, recovery
- hybrid redundancy techniques
  - static + dynamic
  - fault masking + reconfiguration

Active HW redundancy

- dynamic redundancy
  - actions required for correct result
    - detection, localization, containment, recovery
  - no fault masking
  - does not attempt to prevent faults from producing errors within the system

Active HW redundancy

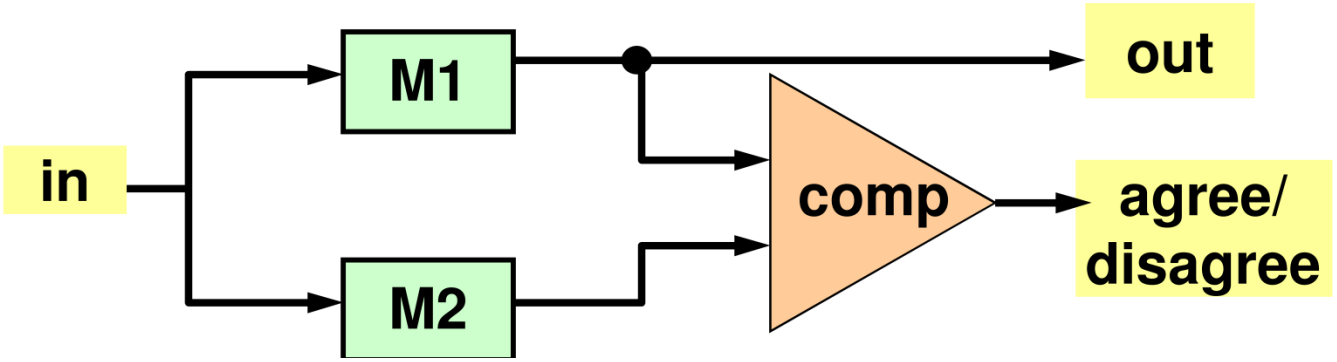
- most common in applications that can tolerate temporary erroneous results
  - satellite systems
  - preferable to have temporary failures that high degree of redundancy

Active HW redundancy

- types of active redundancy:
  - duplication with comparison
  - standby sparing
  - pair-and-a-spare

Duplication with comparison

- Two identical modules perform the same computation in parallel and their results are compared



- The duplication concept can only detect faults, not tolerate them
  - there is no way to determine which module is faulty

Duplication with comparison

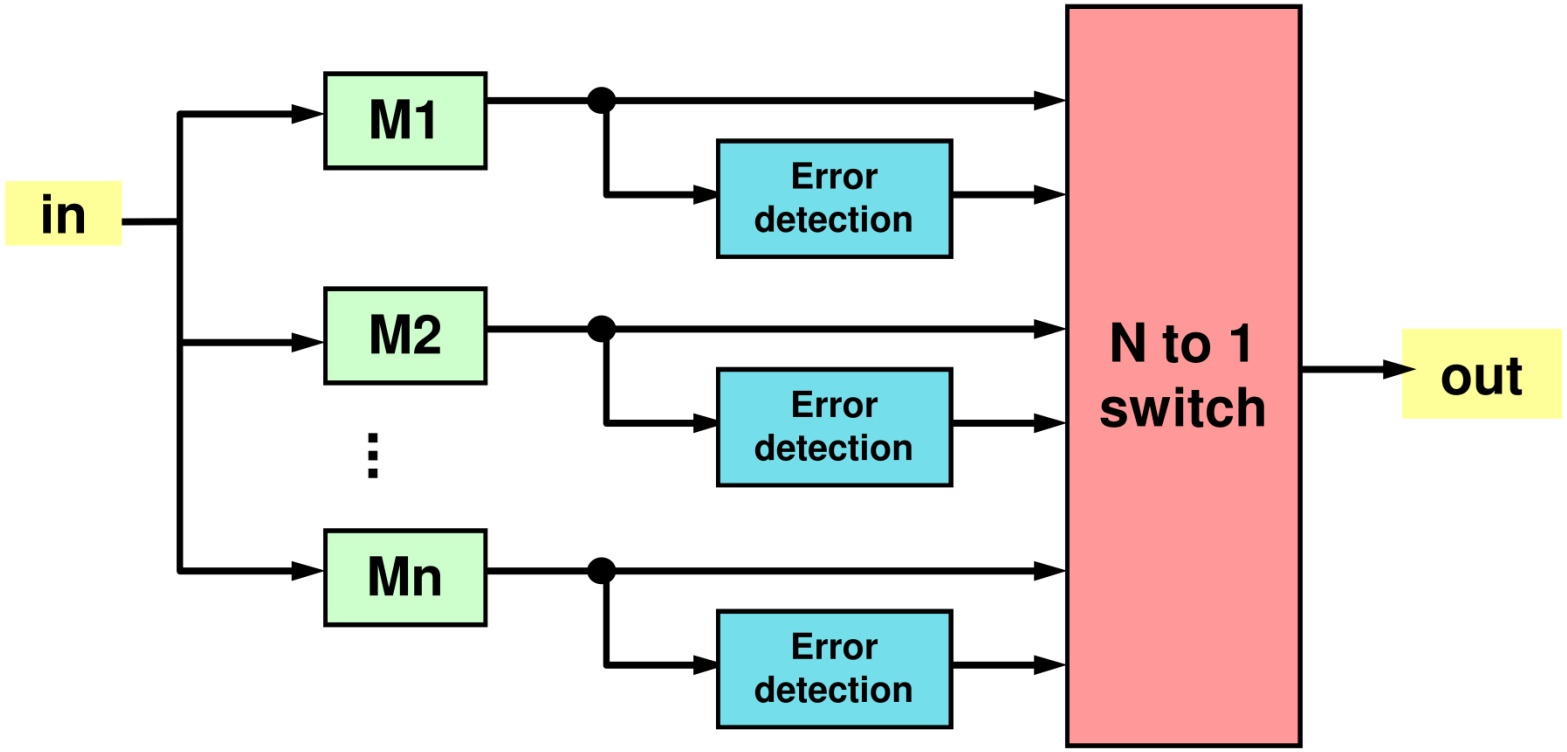
- Problems:
  - if there is a fault on input line, both modules will receive the same erroneous signal and produce the erroneous result
  - comparator may not be able to perform an exact comparison
    - synchronization
    - no exact matching
  - comparator is a single point of failure

Implementation of comparator

- In hardware:
  - a bit-by-bit comparison can be done using two-input exclusive-or gates
- In software:
  - a comparison can be implemented as a COMPARE instruction
    - » commonly found in instruction sets of almost all microprocessors

Standby sparing

- One module is operational and one or more serve as stand-bys, or spares

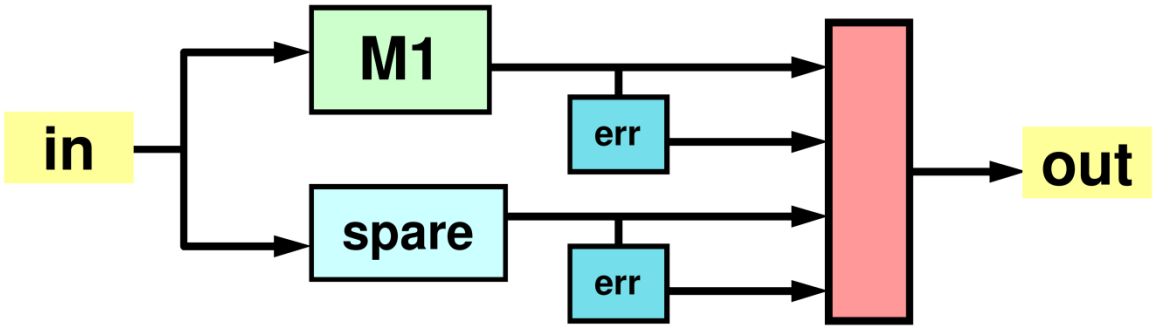


Standby sparing

- error detection is used to determine when a module has become faulty
  - for example, acceptance test
- error location is used to determine which module is faulty
- faulty module is removed from operation and replaced with a spare

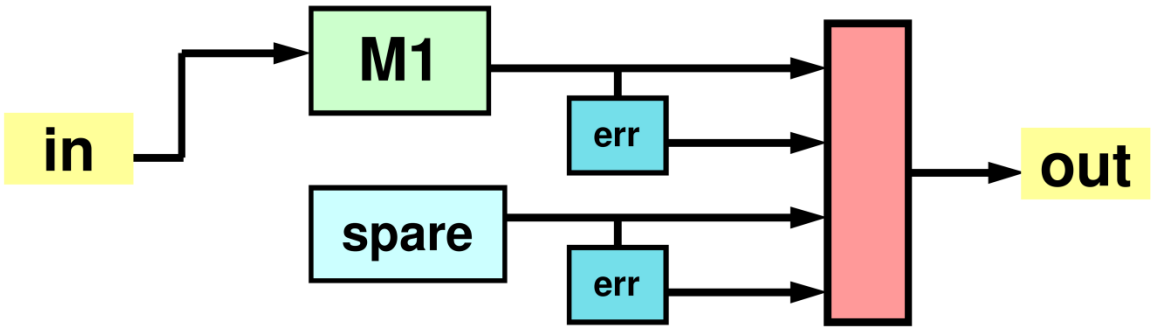
Hot standby sparing

- In hot standby sparing, spares operate in synchrony with on-line module and are prepared to take over any time



Cold standby sparing

- In cold standby sparing, spares are unpowered until needed to replace a faulty module

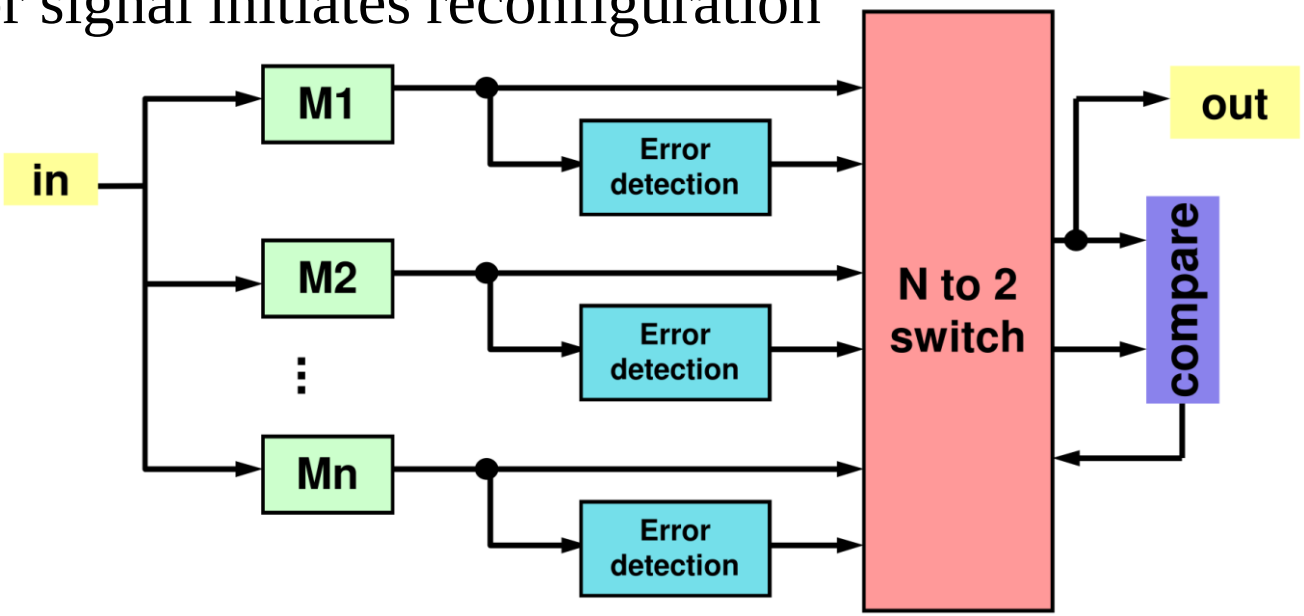


+ and - of cold standby sparing

- (-) time is required to bring the module to operational state
  - time to apply power to spare and to initialize it
  - not desirable in applications requiring minimal reconfiguration time (control of chemical reactions)
- (+) spares do not consume power
  - desirable in applications where power consumption is critical (satellite)

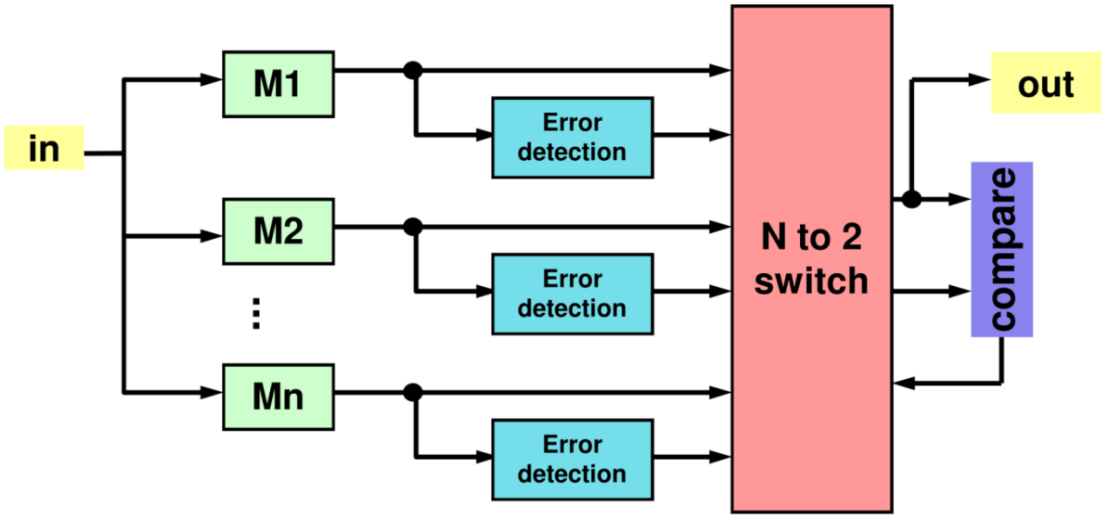
Pair-and-a-spare technique

- Combines standby sparing and duplication with comparison
- like standby sparing, but two instead of one modules are operated in parallel at all times
  - their results are compared to provide error detection
  - error signal initiates reconfiguration



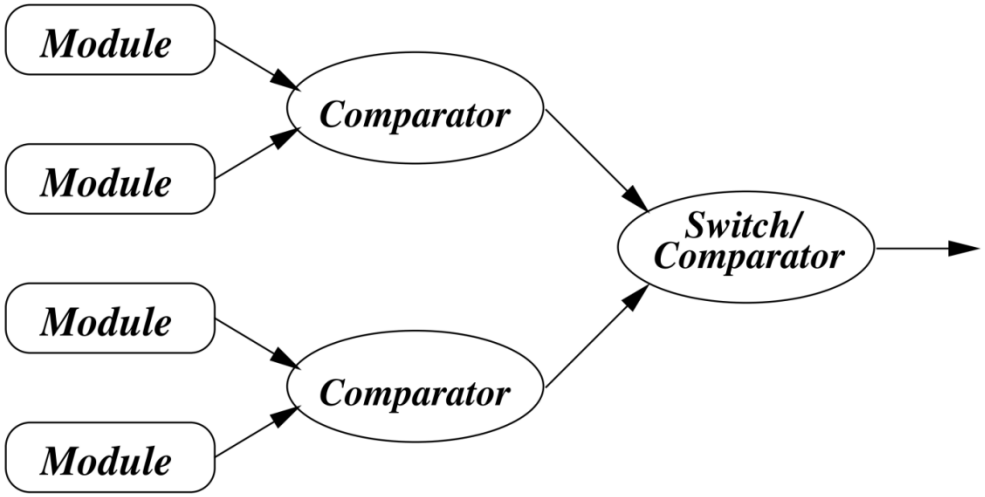
Pair-and-a-spare technique

- As long as two selected outputs agree, the spares are not used
- If they disagree, the switch uses error reports to locate the faulty module and to select the replacement module
  - acceptance tests
  - forward recovery



Another pair-and-spare technique

- modules are grouped in pairs
- each pair has a comparator that checks if the two outputs are equal (or sufficiently close).



Reference

- Slides:
  - Elena Dubrova, Fault-Tolerant Design, Chapter 4
  - Elena Dubrova, Design of Fault Tolerant Systems Course Slides (Lecture 4)